

PrivaMed

Anonimización inteligente de radiografías médicas

Memoria Técnica

Hackathon de Computer Vision — Reto Tecnológico

IABiomed — Universidad de León

Junio 2026

Francisco Javier Luengo Gutiérrez

1. Planteamiento y objetivo

PrivaMed resuelve un problema real en el ámbito sanitario: las radiografías médicas contienen texto superpuesto con datos identificativos del paciente (nombre, DNI, edad, fecha y hora del estudio) que impiden compartir las imágenes para investigación o docencia sin violar el RGPD. La solución habitual — revisión manual por un técnico — es lenta, propensa a errores y no escala.

El planteamiento fue construir un pipeline end-to-end completamente automatizado: un modelo de detección de objetos localiza las regiones de texto sensible en la radiografía, y un módulo de redacción las destruye de forma irreversible sobrescribiendo los píxeles con valor cero (negro). No se usa blur ni overlay porque ambos son reversibles con técnicas de procesamiento de imagen. El resultado es una imagen médicamente útil pero legalmente segura.

2. Arquitectura del sistema

El sistema se despliega con Docker Compose en tres servicios desacoplados:

API REST (FastAPI, puerto 8010). Es el núcleo de procesamiento. Carga el modelo YOLOv8 en memoria al arrancar y expone endpoints para detección (/detect, devuelve JSON con coordenadas y clases), previsualización (/preview, imagen con bounding boxes coloreados), anonimización individual (/anonymize, devuelve PNG) y por lote (/anonymize/batch, devuelve ZIP). La documentación interactiva está disponible en /docs (Swagger). La imagen original nunca se almacena: el procesamiento ocurre en memoria y el resultado se devuelve directamente al cliente.

Frontend (Streamlit, puerto 8501). Dashboard visual que consume la API internamente. Muestra el estado del modelo con sus métricas de evaluación, permite subir imágenes individuales o en lote, presenta comparativa visual detecciones vs. resultado anonimizado, y ofrece descarga directa. La barra lateral permite configurar el umbral de confianza, margen de redacción y qué clases redactar (nombre, ID y edad por defecto; fecha y hora opcionales).

Entrenamiento (perfil Docker bajo demanda). Script Python independiente que entrena el modelo YOLOv8 sin dependencia de Streamlit, ejecuta validación final, y persiste tanto los pesos como un archivo de metadatos JSON con métricas, fecha y configuración del entrenamiento.

La separación API/frontend permite que cualquier sistema externo (Sistema de Información Hospitalaria, script de procesamiento batch, otra interfaz) consuma la funcionalidad de anonimización directamente vía REST, sin depender del dashboard.

3. Modelo de detección

Utilicé YOLOv8n (variante nano) como modelo base. El dataset proporcionado contiene 400 radiografías anotadas en formato YOLO (320 train, 80 val) con 5 clases: name, id, age, date y time.

El entrenamiento se realizó durante 50 épocas con imágenes de 640×640 píxeles, batch size 16 y optimizador AdamW automático, sobre CPU Intel i7-12700K. Los resultados de validación fueron: mAP@50 de 98.71%, mAP@50-95 de 90.73%, precisión del 99.38% y recall del 98.29%. La clase con menor rendimiento fue "time" (mAP@50-95: 84.7%), coherente con el hecho de que es la clase con menos instancias en el dataset (24 vs. 80 del resto). Las clases críticas para la privacidad (name, id, age) superan el 89.5% en todas las métricas.

4. Dificultades técnicas

Durante el desarrollo surgieron varias dificultades técnicas, principalmente relacionadas con la preparación del dataset y la integración del entrenamiento dentro del entorno contenerizado:

4.1. Desalineamiento imagen-etiqueta: El dataset proporcionaba imágenes con sufijo `_annotated` en el nombre de archivo, pero los archivos de etiquetas no lo tenían. YOLO requiere correspondencia exacta de nombre entre imagen y label. Esto causaba que el entrenamiento viera «320 backgrounds, 0 images». Lo resolví renombrando las imágenes para eliminar el sufijo.

4.2. Cachés corruptas de YOLO: Ultralytics genera archivos `.cache` al escanear las etiquetas. La primera ejecución (con nombres desalineados) creó cachés que registraban 0 etiquetas. Reconstruir el contenedor Docker no eliminaba estas cachés porque se generan en runtime. Implementé limpieza automática de cachés tanto en el Dockerfile como en el script de entrenamiento.

4.3. Formato inválido de data.yaml: El archivo de configuración del dataset contenía marcadores de código markdown que impedían su parseo como YAML válido. Además, las rutas relativas no resolvían correctamente dentro del contenedor Docker. Lo corregí eliminando los marcadores y añadiendo `path: /app` para resolución absoluta.

4.4. Streamlit interrumpe el entrenamiento: El file watcher de Streamlit detectaba los archivos de checkpoint que YOLO genera durante el entrenamiento y reiniciaba la aplicación, matando el proceso. La solución definitiva fue extraer el entrenamiento a un servicio Docker completamente separado, ejecutable bajo demanda con `docker compose --profile train run --rm train`.

4.5. Validación con tensores vacíos: Durante las primeras épocas, cuando el modelo aún no produce detecciones válidas, la función de validación de Ultralytics intentaba ejecutar `torch.cat()` sobre una lista vacía, provocando un crash. Implementé manejo de errores en el trainer para capturar esta excepción y continuar el entrenamiento.

5. Herramientas de IA utilizadas

YOLOv8 (Ultralytics v8.3.145) es el framework de detección de objetos que constituye el núcleo del pipeline. Empleé transfer learning partiendo de pesos preentrenados en COCO, fine-tuneados con el dataset específico de radiografías. La elección de la

variante nano fue deliberada para permitir inferencia eficiente en CPU sin sacrificar precisión en un dominio donde los patrones de texto son visualmente consistentes. Claude Code (Anthropic) lo utilicé como herramienta de apoyo para la redacción de código una vez diseñada la arquitectura y definido el flujo de procesamiento.

6. Stack tecnológico

Python 3.11, YOLOv8 (Ultralytics), FastAPI + Uvicorn, Streamlit, OpenCV, PyTorch 2.12, Docker Compose.

7. Instrucciones de ejecución

Requisitos: Docker y Docker Compose instalados.

- 1. Entrenar el modelo:** `docker compose --profile train run --rm train`
- 2. Levantar la aplicación:** `docker compose up --build`
- 3. Acceder al dashboard:** `http://localhost:8501`
- 4. Documentación API:** `http://localhost:8010/docs`